

MULTI-VPC WEB INFRASTRUCTURE

Deployment Guide

Bastion VPC · Transit Gateway · App VPC · ALB · Auto Scaling · S3 · IAM
Two VPCs, one Region · us-east-1

Prepared by

Muhammad Abdullah Muzaffar

DevOps & Cloud Engineer

A hands-on reference build for multi-VPC networking, Transit Gateway routing, and private compute at scale.

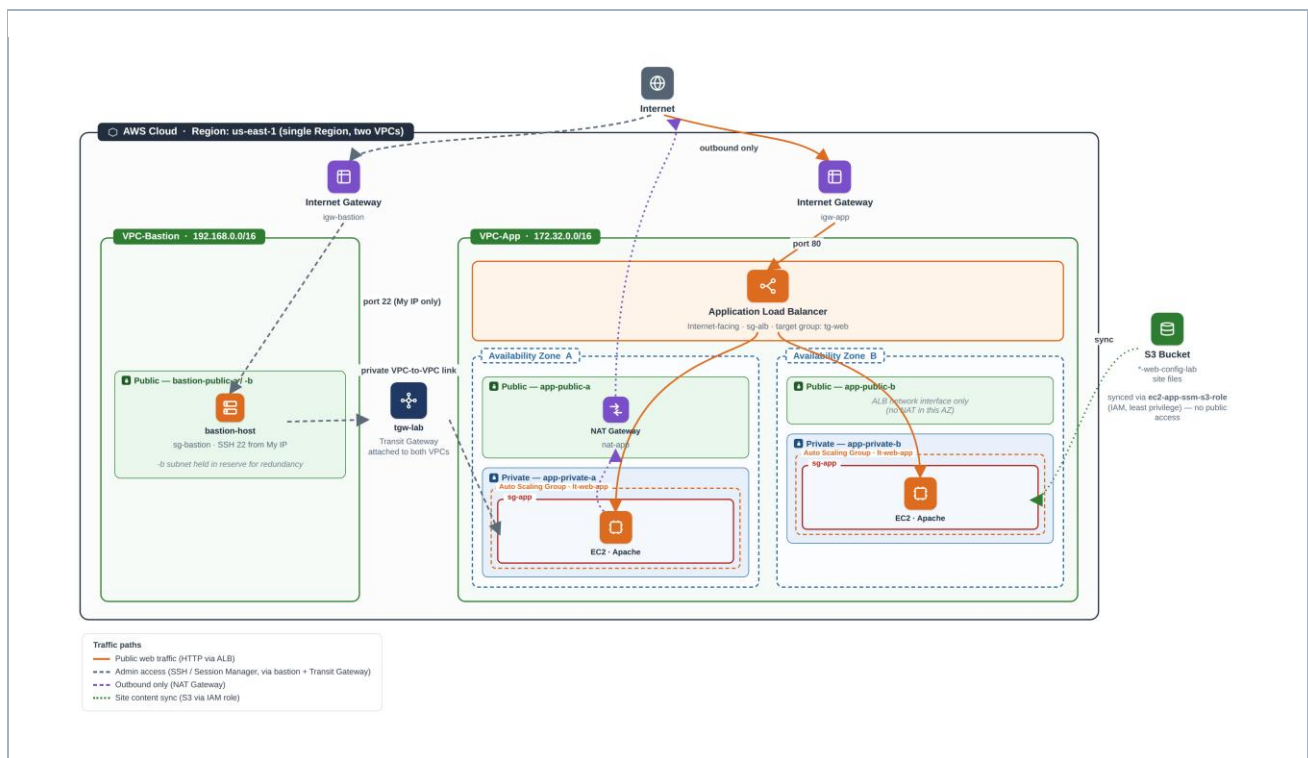
Project Overview

This build is a multi-VPC AWS network: two separate Virtual Private Clouds, kept deliberately isolated from each other at the routing level, then reconnected on purpose through a Transit Gateway. It is a step up from a single-VPC design — the same public/private separation as before, but now split across VPC boundaries, which is how most real organizations actually grow their AWS footprint (a shared services VPC, a bastion VPC, several application VPCs, all peered through a hub).

One small VPC exists purely to hold a bastion host — the single, tightly-restricted front door for admin access. A second, larger VPC holds the actual application: a public tier with a Load Balancer and a NAT Gateway, and a private tier of web servers in an Auto Scaling Group. Neither VPC can reach the other by default — a Transit Gateway, with explicit routes on both sides, is what makes the bastion's SSH path into the private app servers possible at all. Everything the web servers need at boot — the site content — comes from a private S3 bucket, read through an IAM role rather than any public URL.

What this build uses:

- Compute — Amazon EC2 (Amazon Linux, Apache httpd) baked into a Golden AMI, run through a Launch Template and an Auto Scaling Group
- Networking — two VPCs (a Bastion VPC and an App VPC) joined by a Transit Gateway, each with its own public/private subnets across 2 Availability Zones
- Load balancing — an internet-facing Application Load Balancer in front of the private web tier
- Storage — Amazon S3 holds the site content, synced to instances at boot via an IAM role (least privilege, no public access)
- Admin access — a bastion host in its own VPC, reachable only via SSH from a single trusted IP, plus Session Manager as a keyless alternative
- Observability — VPC Flow Logs shipped to CloudWatch for both VPCs



Complete infrastructure — the multi-VPC web build

Solid = public web traffic · dashed grey = admin access via bastion + Transit Gateway · dashed purple = outbound-only · dotted green = content sync

Everything from here follows the build in phase order — network first (both VPCs, then the link between them), then the compute and content layers, then the public-facing pieces, then validation and cleanup.

About This Guide

This is a reference build, not a record of a specific run — so instead of an incident log, the teaching points from the original lab are kept as blue “Learning” notes right where they’re relevant, and common first-time mistakes are flagged the same way as you go, like the box below.

Learning — boxes like this one explain **why** a step is done a certain way, not just what to click.

⚠ Common pitfall

boxes like this one flag a mistake that’s easy to make at this exact step — see also “Beginner Pitfalls” near the end for the full list.

Before you start:

- Sign in to the AWS Console and pick one Region (e.g. us-east-1) — stay there for everything in this build.
- A free-tier eligible account is fine, but expect some cost: the NAT Gateway, the ALB, and any unattached Elastic IP are the main bill drivers.
- Create and download an EC2 key pair early: EC2 → Key pairs → Create.
- Keep a running note of every resource name and ID as you create it — it makes cleanup, and troubleshooting, far easier.

Learning — a Region is sticky. Resources in one Region can’t see resources in another unless something (like a Transit Gateway peering, or S3 cross-region replication) is deliberately set up to cross that boundary.

PHASE 0 Golden AMI (Pre-Deployment)

Before any Auto Scaling is involved, bake the common software into a custom AMI, so every instance the ASG ever launches starts up already configured — instead of installing packages on every single boot.

Name	golden-ami-builder
AMI	Amazon Linux 2023 (or Amazon Linux 2)
Instance type	t3.micro
Key pair	your key
Network	the default VPC is fine for this one-off bake step
Security group	SSH 22 from My IP only
IAM role	AmazonSSMManagedInstanceCore (S3 read comes later, in Phase 7)

Launch it, then connect with Session Manager or SSH, and run:

```
sudo dnf update -y # or: sudo yum update -y
sudo dnf install -y httpd git
sudo systemctl enable httpd
sudo systemctl start httpd
aws --version
sudo dnf install -y amazon-cloudwatch-agent
```

The CloudWatch Agent is optional (for memory / custom metrics) — the SSM Agent needed for Session Manager is normally already preinstalled on Amazon Linux.

Once the software is installed and working: EC2 → Actions → Image and templates → Create image. Name it golden-ami-web, wait for it to reach Available, then terminate golden-ami-builder — its job is done.

Learning — an AMI is a snapshot of an OS plus whatever's installed on it. The Launch Template and the Auto Scaling Group reuse this one image for every new instance, so every server that ever gets launched is identical from the first second it boots.

PHASE 1 Bastion VPC (192.168.0.0/16)

1.1 Create the VPC

VPC → Your VPCs → Create VPC → VPC only.

Name	vpc-bastion
IPv4 CIDR	192.168.0.0/16
DNS hostnames	Enabled

1.2 Public subnets

Name	AZ	CIDR	Type
bastion-public-a	AZ-a	192.168.1.0/24	Public
bastion-public-b	AZ-b	192.168.2.0/24	Public

One public subnet is technically enough for a single bastion host — a second is good practice and costs nothing extra to reserve.

1.3 Internet Gateway

Create igw-bastion and attach it to vpc-bastion.

1.4 Public route table

Route	Target
192.168.0.0/16	local
0.0.0.0/0	igw-bastion

Associate rtb-bastion-public with both public subnets.

Learning — a subnet is “public” purely because its route table sends 0.0.0.0/0 to an Internet Gateway — nothing about its name or its CIDR makes it public on its own.

PHASE 2 App VPC (172.32.0.0/16)

Create vpc-app with CIDR 172.32.0.0/16, then create igw-app and attach it.

Name	AZ	CIDR	Role
app-public-a	AZ-a	172.32.1.0/24	ALB + NAT

Name	AZ	CIDR	Role
app-public-b	AZ-b	172.32.2.0/24	ALB
app-private-a	AZ-a	172.32.11.0/24	ASG
app-private-b	AZ-b	172.32.12.0/24	ASG

Create `rtb-app-public`, associate both public subnets, and route `0.0.0.0/0` → `igw-app`.

2.1 NAT Gateway

VPC → Elastic IPs → Allocate, then VPC → NAT gateways → Create:

Name	nat-app
Subnet	app-public-a
Elastic IP	the EIP just allocated

Create `rtb-app-private`, associate both private subnets, and route `0.0.0.0/0` → `nat-app`.

Traffic	Path
Private instance → internet (outbound)	Private subnet → NAT Gateway → Internet Gateway
Internet → private instance (inbound)	Never direct — users hit the ALB, not the NAT Gateway

PHASE 3 Transit Gateway

VPC → Transit gateways → Create. Name it `tgw-lab` — the defaults are fine for a build like this one.

Create two attachments — one for `vpc-bastion`, one for `vpc-app` — and wait until both show Available.

VPC	Destination	Target
Bastion	172.32.0.0/16	Transit Gateway
App	192.168.0.0/16	Transit Gateway

Add these routes to each VPC's route tables so traffic addressed to the other VPC's CIDR actually gets sent to the Transit Gateway attachment.

Learning — a Transit Gateway is a hub, not magic — attaching two VPCs to it doesn't connect them by itself. Without an explicit route in each VPC's route table pointing the other VPC's CIDR at the Transit Gateway, the two VPCs stay completely isolated even though they're both attached to the same hub.

PHASE 4 VPC Flow Logs + CloudWatch

Create a CloudWatch log group named `/aws/vpc/flowlogs`.

Create an IAM role trusted by `vpc-flow-logs.amazonaws.com`, with permission to write to CloudWatch Logs.

For each VPC (`vpc-bastion` and `vpc-app`): VPC → select the VPC → Flow logs → Create.

Filter	All
Destination	CloudWatch Logs
Log group	<code>/aws/vpc/flowlogs</code>

IAM role	the flow-logs role created above
-----------------	----------------------------------

Learning — *Flow Logs record accept/reject decisions at the network-interface level. They're the first place to look when a security group or NACL is silently dropping traffic and it isn't obvious why.*

PHASE 5 Bastion Host

Security group	sg-bastion
Inbound	SSH · TCP 22
Source	My IP only
Instance name	bastion-host
AMI	Amazon Linux
Instance type	t3.micro
Subnet	bastion-public-a
Public IP	Enable (or attach an Elastic IP)
Key pair	your key

Learning — *the bastion is the one controlled door from the internet into this whole design. Every other instance stays private precisely so this is the only thing anyone ever needs to expose.*

PHASE 6 S3 Config Bucket

Bucket name	globally unique, e.g. yourname-web-config-lab
Block Public Access	ON
Content	the site files — at minimum an index.html, or a full static site

Learning — *a private S3 bucket paired with an IAM role on the instance is the better pattern here — not a public bucket. The instance authenticates as itself; the content is never reachable by a random URL.*

PHASE 7 IAM Role for App Instances

Permission	Purpose
AmazonSSMManagedInstanceCore	Session Manager access, no SSH key needed
Custom S3 policy	Least-privilege read of your bucket only

Custom policy JSON (replace YOUR_BUCKET_NAME):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME",
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*"
    }
  ]
}
```

Role name: ec2-app-ssm-s3-role.

⚠ Common pitfall

it's tempting to reach for the managed AmazonS3FullAccess policy to save time — don't. It grants access to every bucket in the account, not just this one; the scoped custom policy above is barely more effort.

PHASE 8 Application Security Groups

Create sg-alb first: inbound HTTP 80 from 0.0.0.0/0 (add 443 later if HTTPS is added).

Security Group	Port	Source
sg-app	22	sg-bastion
sg-app	80	sg-alb

Learning — *referencing another security group as the source (instead of an IP range) is the cleaner pattern — sg-app's rules describe **who** is allowed in (the ALB, the bastion) rather than **where** they happen to be, and the rule stays correct even if IPs change.*

PHASE 9 Launch Template

Name	lt-web-app
AMI	golden-ami-web
Instance type	t3.micro
Key pair	your key
Security group	sg-app
IAM instance profile	ec2-app-ssm-s3-role
Subnet	leave unset — the Auto Scaling Group chooses it

User data:

```
#!/bin/bash
set -xe
systemctl enable httpd || true
systemctl start httpd || true
aws s3 sync s3://YOUR_BUCKET_NAME/ /var/www/html/
# OR clone from Git:
# yum install -y git
# git clone https://github.com/YOUR_REPO.git /tmp/app
# cp -r /tmp/app/* /var/www/html/
systemctl restart httpd
```

Learning — *user data runs exactly once, on an instance's very first boot. Pairing a Golden AMI with this thin a user-data script is what makes scale-out fast and reliable — there's almost nothing left to install or configure by the time this script runs.*

PHASE 10 Target Group + ALB + Auto Scaling Group

10.1 Target group

Type	Instances
Protocol / port	HTTP / 80
VPC	vpc-app
Health check	/ or /index.html
Name	tg-web

10.2 Application Load Balancer

Scheme	Internet-facing
Subnets	app-public-a and app-public-b
Security group	sg-alb
Listener	HTTP :80 → tg-web

10.3 Auto Scaling Group

Launch template	lt-web-app
VPC	vpc-app
Subnets	app-private-a and app-private-b
Target group	tg-web
Desired / Min	2 / 2
Max	4
Health checks	ELB + EC2

Wait until two targets show healthy.

Learning — *the three pieces split cleanly: the Auto Scaling Group owns capacity (how many instances exist), the ALB owns traffic distribution (which instance gets the next request), and the Target Group is the health/routing glue between them. Spreading the private subnets across two AZs is what actually delivers the high availability.*

PHASE 11 Route 53 (Optional)

- Route 53 → Hosted zones → your zone.
- Create an A record with Alias to the ALB (preferred), or a CNAME to the ALB's DNS name.
- Browse to <http://your.domain>.

Without a domain, just open the ALB's DNS name directly from EC2 → Load Balancers.

PHASE 12 Validation

12.1 Bastion → private instance

```
# From your PC
ssh -i your-key.pem ec2-user@BASTION_PUBLIC_IP

# From the bastion to a private ASG instance
ssh -i your-key.pem ec2-user@172.32.x.x
```

You may need to copy the key onto the bastion carefully, or use SSH agent forwarding, to make the second hop work.

12.2 Session Manager

EC2 → select a private instance → Connect → Session Manager. This works purely on AmazonSSMManagedInstanceCore, the SSM Agent, and outbound connectivity to AWS endpoints via the NAT Gateway — no key, no open port 22 required at all.

12.3 Public web test

Browser → the ALB's DNS name (or your Route 53 domain) → the HTML page should load.

Key Learning Checklist

Concept	Takeaway
Public vs. private subnet	Defined entirely by its route (IGW vs. NAT) — never by its name
Bastion	Controlled admin entry point; restrict SSH to your own IP only
NAT Gateway	Outbound-only, and it bills hourly for as long as it exists
Transit Gateway	Multi-VPC private connectivity — needs explicit routes on both sides
Launch Template	The modern replacement for the older Launch Configuration
ASG Min/Max	Elasticity and high availability spread across Availability Zones
ALB + Target Group	Layer-7 entry point into private compute
IAM on EC2	Prefer Session Manager plus a narrow S3 policy; avoid S3 Full Access
Flow Logs	Network visibility for both security review and troubleshooting
Golden AMI	Bake once, scale out many times identically

Suggested Learning Order

Day	Goal
Day 1	VPC-Bastion + Internet Gateway + bastion EC2 + SSH
Day 2	VPC-App + public/private subnets + NAT + Transit Gateway routes
Day 3	Golden AMI + S3 + IAM + Launch Template
Day 4	ALB + Target Group + ASG + browser test

Day	Goal
Day 5	Flow Logs + Session Manager + optional Route 53
Always	Cleanup

Cleanup — Prevent Unexpected Charges

Delete in this order — dependencies matter, and the NAT Gateway and any Elastic IP are the two easiest things to forget.

#	Resource	Note
1	Auto Scaling Group	Set desired capacity to 0 first, then delete the ASG
2	Application Load Balancer	Then the Target Group
3	EC2 bastion	Then the Launch Template
4	NAT Gateway	Wait for full deletion, then release its Elastic IP
5	Transit Gateway attachments	Then the Transit Gateway itself
6	Remaining ENIs	If any are left over from deleted resources
7	Security Groups	Only after every instance using them is gone
8	Subnets → route tables → Internet Gateways → VPCs	In that order, for both VPCs
9	CloudWatch log groups	/aws/vpc/flowlogs and any others created
10	S3 objects + bucket	Empty the bucket before deleting it
11	IAM roles / policies	Any created specifically for this build
12	AMI + snapshots	golden-ami-web, if no longer needed

Beginner Pitfalls

The mistakes below are the ones this design is most likely to surface on a first attempt — each one is a one-line fix once you know to look for it.

⚠ Common pitfall

Forgetting the 0.0.0.0/0 → NAT route on the private route table blocks yum, S3, and SSM connectivity for every private instance, with no obvious error message pointing at the cause.

⚠ Common pitfall

Putting the ASG in public subnets technically works, but misses the entire point of the private-tier exercise.

⚠ Common pitfall

Putting the ALB in private subnets prevents it from being internet-facing at all — an internet-facing ALB needs public subnets.

⚠ Common pitfall

Opening sg-app's port 22 to 0.0.0.0/0 defeats the purpose of having a bastion in the first place.

⚠ Common pitfall

Leaving the NAT Gateway running overnight (or over a weekend) is one of the most common sources of a surprise AWS bill in a lab like this.

Final Project Outcome

The completed build demonstrates a genuine multi-VPC AWS web architecture: a public bastion VPC connects to a private application VPC through a Transit Gateway. The application VPC runs a public ALB and NAT tier alongside a private, Auto-Scaled fleet of web servers. S3 and IAM deliver site content with least privilege, Session Manager provides keyless administration, and VPC Flow Logs give network-level visibility across both VPCs.

End of document.